EXPERIMENT 1

Student Name: PRIYANSHU **UID: 24BAI70700**
Branch: AIT-CSE-AIML **Section/Group: 24AIT_KRG1_B**
Semester: 5
Subject Name: ADBMS **Subject Code: 24CSP-310**

QUESTION A):
Write an SQL solution to calculate the total number of bank accounts belonging to each salary category. The classification categories are:

1. "Low Salary": All monthly salaries strictly less than $20,000.
2. "Average Salary": All monthly salaries in the inclusive range [$20,000, $50,000].
3. "High Salary": All monthly salaries strictly greater than $50,000. The final result table must report all three categories, even if a category contains zero accounts.

SOLUTION:

QUERY:

Experiment 1/postgres@PostgreSQL 14

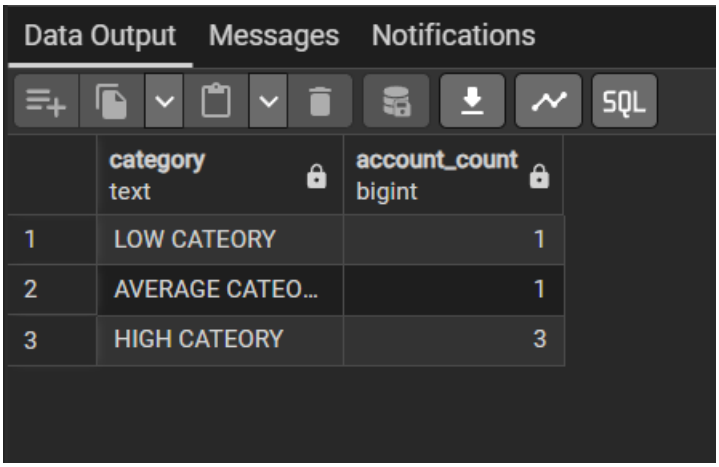
No limit

Query Query History

```sql
CREATE TABLE Accounts (
    account_id INT PRIMARY KEY,
    income INT NOT NULL
);

INSERT INTO Accounts (account_id, income) VALUES
(3, 108939),
(2, 12747),
(8, 87709),
(6, 91738),
(7, 45169);

SELECT  'LOW CATEORY' AS  CATEGORY,
        COUNT(ACCOUNT_ID)   AS ACCOUNT_COUNT
FROM ACCOUNTS
WHERE INCOME <20000

UNION ALL

SELECT  'AVERAGE CATEORY' AS  CATEGORY,
        COUNT(ACCOUNT_ID)   AS ACCOUNT_COUNT
FROM ACCOUNTS
WHERE INCOME >20000 AND INCOME <=50000

UNION ALL

SELECT  'HIGH CATEORY' AS  CATEGORY,
        COUNT(ACCOUNT_ID)   AS ACCOUNT_COUNT
FROM ACCOUNTS
WHERE INCOME >50000;
```

EXPERIMENT 1

OUTPUT:



	category text	account_count bigint
1	LOW CATEORY	1
2	AVERAGE CATEO...	1
3	HIGH CATEORY	3

QUESTION (B)

Design an Employee Management System database architecture by implementing the following operations sequentially:

1. Create and populate structural tracking tables for Departments, Employees, and Salaries.
2. Query the system using multi-table Joins to pull clear department breakdowns of employees, department and salaries, apply a 10% salary enhancement across all members of the HR department, and delete any individual salary record dropping strictly below $30,000.
3. List all remaining active employees whose personal salary exceeds the overall company-wide average salary.

SOLUTION:

QUERY:

EXPERIMENT 1

Experiment 1/postgres@PostgreSQL 14

No limit

Query | Query History

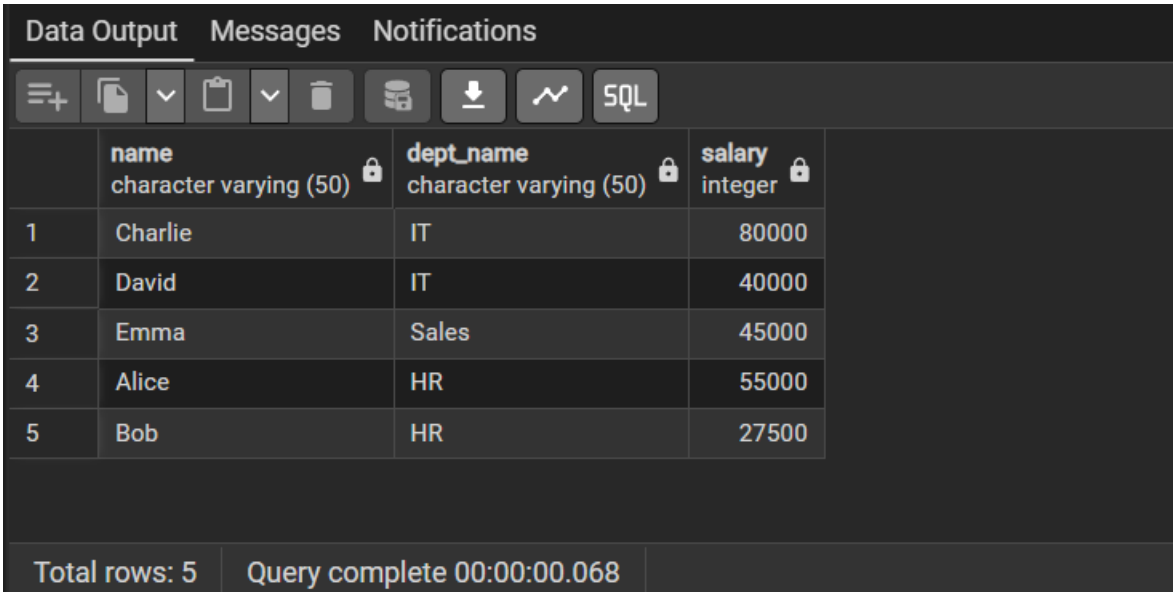
```sql
CREATE TABLE Departments (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(50) NOT NULL
);

CREATE TABLE Employees (
    emp_id INT PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    dept_id INT REFERENCES Departments(dept_id)
);

CREATE TABLE Salaries (
    emp_id INT PRIMARY KEY REFERENCES Employees(emp_id) ON DELETE CASCADE,
    salary INT NOT NULL
);


INSERT INTO Departments (dept_id, dept_name) VALUES
(10, 'HR'), (20, 'IT'), (30, 'Sales');

INSERT INTO Employees (emp_id, name, dept_id) VALUES
(1, 'Alice', 10), (2, 'Bob', 10), (3, 'Charlie', 20), (4, 'David', 20), (5, 'Emma', 30);

INSERT INTO Salaries (emp_id, salary) VALUES
(1, 50000), (2, 25000), (3, 80000), (4, 40000), (5, 45000);
```

```sql
    SELECT E.*, D.*, S.*
    FROM
    EMPLOYEES AS E
    JOIN
    DEPARTMENTS AS D
    ON
    E.DEPT_ID = D.DEPT_ID
    JOIN
    SALARIES AS S
    ON
    E.EMP_ID = S.EMP_ID;

    UPDATE SALARIES
    SET SALARY = SALARY + SALARY * 0.10
    WHERE EMP_ID
    IN
    (
    SELECT E.EMP_ID
    FROM EMPLOYEES AS E
    JOIN
    DEPARTMENTS AS D
    ON
    E.DEPT_ID = D.DEPT_ID
    WHERE D.DEPT_NAME = 'HR'
    )
```

```sql
    SELECT E.NAME, D.DEPT_NAME, S.SALARY
    FROM
    EMPLOYEES AS E
    JOIN
    DEPARTMENTS AS D
    ON
    E.DEPT_ID = D.DEPT_ID
    JOIN
    SALARIES AS S
    ON
    E.EMP_ID = S.EMP_ID;
```

EXPERIMENT 1

OUTPUT:



	name character varying (50)	dept_name character varying (50)	salary integer
1	Charlie	IT	80000
2	David	IT	40000
3	Emma	Sales	45000
4	Alice	HR	55000
5	Bob	HR	27500

Total rows: 5 Query complete 00:00:00.068

QUESTION (C):

A pizza restaurant stores all the available pizza toppings and their ingredient costs in a table called pizza_toppings.

Every customer must choose exactly 3 different toppings to prepare a pizza.

Write a PostgreSQL query to generate every possible 3-topping pizza combination along with its total ingredient cost.

Requirements

Every pizza must contain exactly 3 different toppings.

The same topping cannot appear more than once in a pizza.

Different arrangements of the same toppings should be considered the same pizza.

Example:

Chicken, Pepperoni, Sausage

Pepperoni, Chicken, Sausage

Sausage, Chicken, Pepperoni

These are all the same pizza, so only one should appear.

Display the topping names separated by commas.

Display the total ingredient cost of each pizza.

Sort the output:

First by highest total cost

If two pizzas have the same cost, sort them alphabetically.

EXPERIMENT 1

SOLUTION:

QUERY:

```sql
CREATE TABLE pizza_toppings (
    topping_name VARCHAR(50) PRIMARY KEY,
    ingredient_cost NUMERIC(4,2) NOT NULL
);


INSERT INTO pizza_toppings (topping_name, ingredient_cost) VALUES
('Pepperoni', 0.50),
('Sausage', 0.70),
('Chicken', 0.55),
('Onions', 0.25),
('Extra Cheese', 0.40);


SELECT
    CONCAT(p1.topping_name, ',', p2.topping_name, ',', p3.topping_name) AS pizza,
    (p1.ingredient_cost + p2.ingredient_cost + p3.ingredient_cost) AS total_cost
FROM pizza_toppings AS p1
JOIN pizza_toppings AS p2
    ON p1.topping_name < p2.topping_name
JOIN pizza_toppings AS p3
    ON p2.topping_name < p3.topping_name
ORDER BY
    total_cost DESC
```

OUTPUT:

Data Output | Messages | Notifications

	pizza text	total_cost numeric
1	Chicken,Pepperoni,Sausage	1.75
2	Chicken,Extra Cheese,Sausage	1.65
3	Extra Cheese,Pepperoni,Saus...	1.60
4	Chicken,Onions,Sausage	1.50
5	Chicken,Extra Cheese,Pepper...	1.45
6	Onions,Pepperoni,Sausage	1.45
7	Extra Cheese,Onions,Sausage	1.35
8	Chicken,Onions,Pepperoni	1.30
9	Chicken,Extra Cheese,Onions	1.20
10	Extra Cheese,Onions,Pepperoni	1.15

EXPERIMENT 1

QUESTION (D)

Amazon wants to identify returning customers who made another purchase shortly after their first purchase.
Write a PostgreSQL query to find all users who made another purchase within 1 to 7 days of an earlier purchase.
Requirements

1. Compare purchases made by the same user only.
2. Ignore comparisons where the same transaction is matched with itself.
3. The second purchase must occur after the first purchase.
4. Same-day purchases must not be considered.
5. The second purchase must occur within 7 days of the first purchase.
6. Display only the unique User IDs that satisfy the above conditions.
7. Sort the output in ascending order of user_id.

SOLUTION:
QUERY:

Query | Query History

```
CREATE TABLE amazon_purchases (
    id SERIAL PRIMARY KEY,
    user_id INT NOT NULL,
    item_name VARCHAR(255),
    created_at TIMESTAMP NOT NULL,
    amount DECIMAL(10, 2)
);

INSERT INTO amazon_purchases (user_id, item_name, created_at, amount) VALUES

(101, 'Book', '2026-03-01 10:00:00', 15.99),
(101, 'Pen',  '2026-03-04 14:30:00', 4.50),

(102, 'Phone', '2026-03-01 09:00:00', 699.99),
(102, 'Case',  '2026-03-01 18:00:00', 25.00),

(103, 'Desk', '2026-03-01 11:00:00', 120.00),
(103, 'Lamp', '2026-03-10 15:00:00', 35.00),

(104, 'Shoes', '2026-03-05 08:00:00', 80.00),
(104, 'Socks', '2026-03-06 09:00:00', 12.00);
```

Data Output | Messages | Notifications

```
INSERT 0 8

Query returned successfully in 282 msec.
```

EXPERIMENT 1

```sql
SELECT DISTINCT p1.user_id
FROM amazon_purchases p1
JOIN amazon_purchases p2
  ON p1.user_id = p2.user_id
  AND p1.id != p2.id
  AND p2.created_at > p1.created_at
  AND p2.created_at::date - p1.created_at::date >= 1
  AND p2.created_at::date - p1.created_at::date <= 7
ORDER BY p1.user_id ASC;
```

OUTPUT:

Data Output | Messages | Notifications

Showing rows: 1 to 2 | Page No: 1 of 1

	user_id integer
1	101
2	104